

Questo PDF rappresenta una sintesi essenziale, ma completa, di tutto ciò che è necessario per ottenere il massimo dei voti (30).

I diagrammi riportati di seguito sono stati valutati dal Professor D.Falessi durante l'esame e la successiva correzione, ricevendo un punteggio pari a 30 - tutti i diagrammi che vedrete sono stati valutati con il punteggio di 30/30.

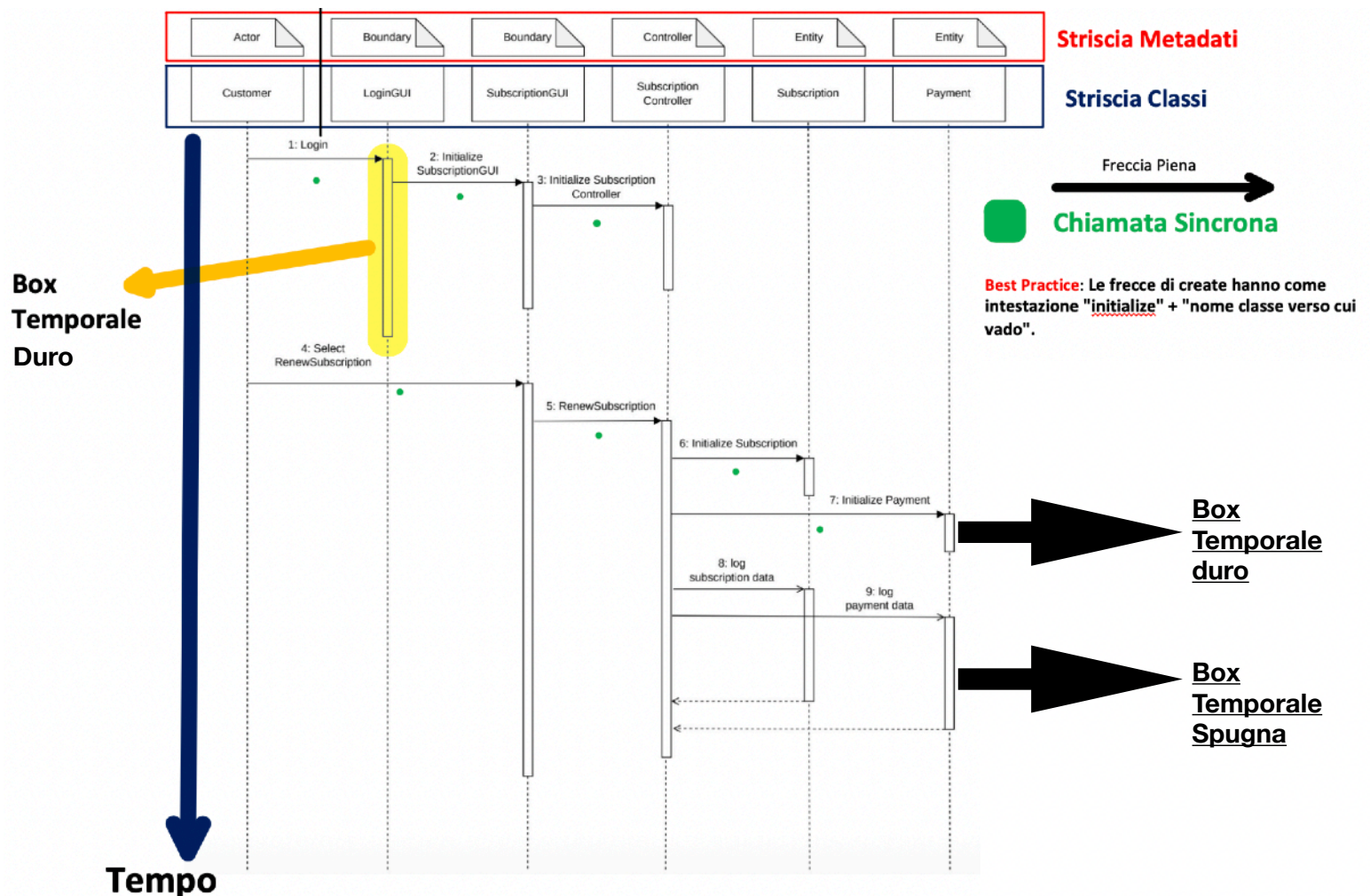
Il diagramma presentato di seguito è stato realizzato dal collega Lacroce.

SEQUENCE DIAGRAM

I diagrammi di sequenza sono diagrammi di interazione che descrivono in dettaglio come vengono eseguite le operazioni: in modo particolare espone le sequenze di chiamata delle classi.

ELEMENTI DI UN SEQUENCE

1. **Striscia Metadati**: è la parte che specifica il tipo delle classi.
2. **Striscia Classi**: sono le classi che partecipano allo svolgimento del caso d'uso.
3. **Attore**: è colui che avvia il caso d'uso.



La best practice vale per tutte le chiamate “create” sincrone tranne per quella che parte dall'attore.

Le operazioni di “create” sono operazioni che danno vita ad una classe.

Ogni classe presa verticalmente avrà una “create”, ossia una freccia che crea il primo box per la prima volta.

Le frecce delle chiamate **sincrone** non possono attraversare box di altre classi: l'output di una freccia con chiamata sincrona è un **muro duro**.

Le frecce delle chiamate **asincrone** possono attraversare box di altre classi: l'output di una freccia con chiamata asincrona è un **muro di spugna**.

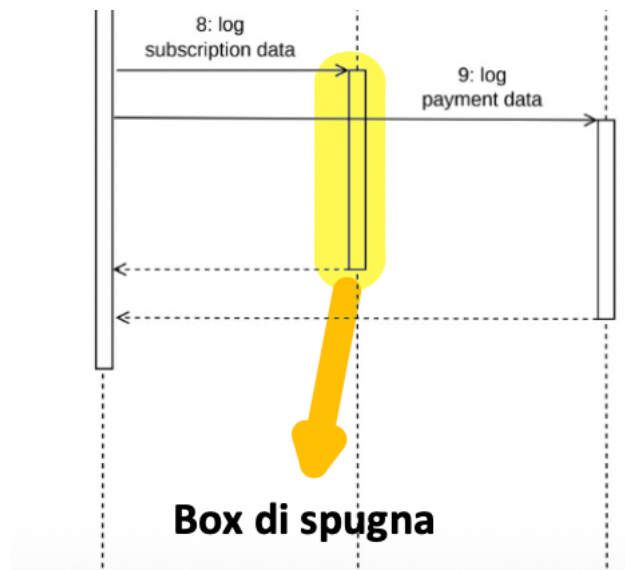
Teorema: Se le chiamate sincrone partono da uno stesso box, per creare una nuova chiamata sincrona bisogna aspettare la morte del box creato dalla chiamata sincrona precedente.

Le chiamate **asincrone** ci piace farle dal controller sulle entity perché si prestano quando devi scrivere o prendere dati da esse. Le chiamate asincrone servono per effettuare operazioni in **parallelo**.

Le create possono essere asincrone (initialize+”nome classe”) per la prima volta.

Teorema: Se le chiamate asincrone partono da uno stesso box, per creare una nuova chiamata asincrona non bisogna aspettare la morte del box creato dalla chiamata asincrona precedente, proprio perché l'output di una freccia con chiamata asincrona è un **muro di spugna, quindi attraversabile**.

Esempio di chiamate asincrone



Quindi, partendo da queste informazioni di base (ma comunque fondamentali), possiamo ora formalizzare ciò di cui stiamo parlando.

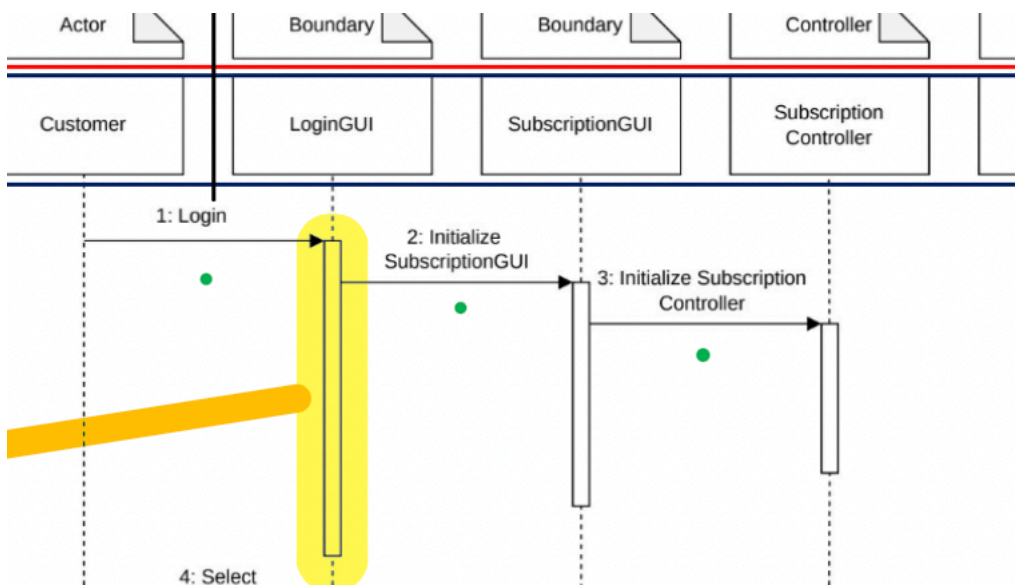
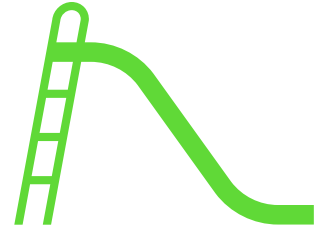
Una chiamata **sincrona** è quella in cui l'oggetto chiamante aspetta la risposta dell'oggetto chiamato prima di proseguire la propria esecuzione. In UML è una **freccia piena con punta triangolare**.

Una chiamata **asincrona** è quella in cui l'oggetto chiamante non attende la risposta, pertanto più azioni possono essere svolte in parallelo. In UML è una **freccia con punta aperta** (una sola stanghetta, non triangolare).

Consigli da esame.

- La parte iniziale è standard: vi è sempre una catena di chiamate create sincrone che parte dall'attore e arriva fino al controller.

Pertanto, sia durante l'esame sia nella redazione della relazione, per andare sul sicuro, procedete in modo semplice e diretto seguendo la consueta catena "a gradino" che parte dall'attore e conduce fino al controller. Di seguito è riportato un esempio.

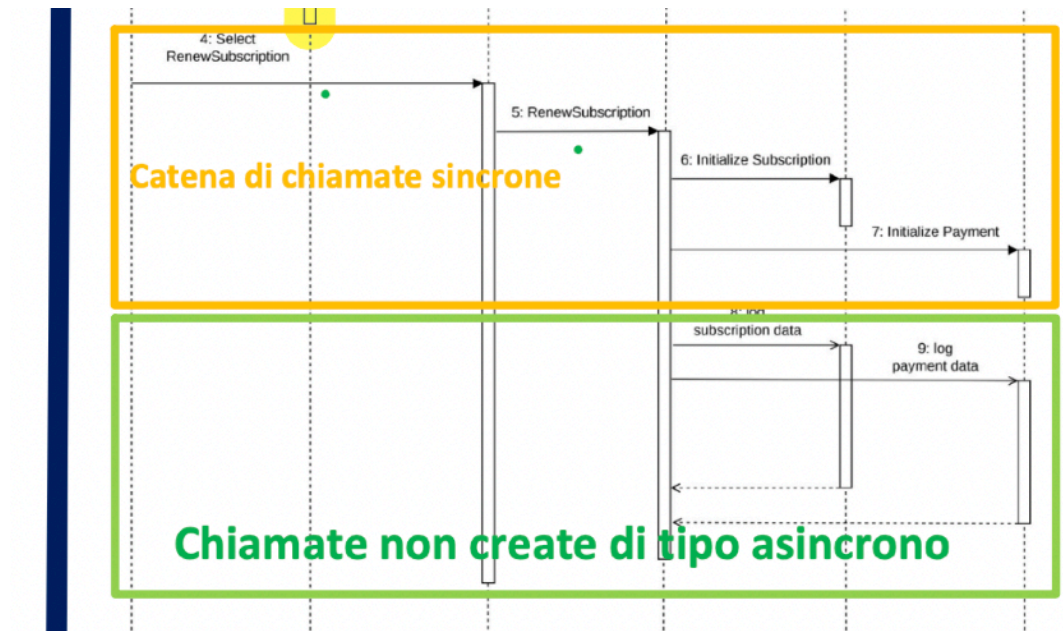


- La seconda parte consiste nell'**azione vera e propria**. Successivamente si effettua una chiamata sincrona, non "create", che parte dall'attore verso la GUI e dalla GUI verso il controller. Successivamente, dal controller alle entità, vengono eseguite chiamate **create** di tipo **sincrono**(init). Dopodiché l'operazione effettiva quindi successivamente, dal controller alle entità, vengono eseguite tutte le chiamate **non create** di tipo asincrono (**log**).

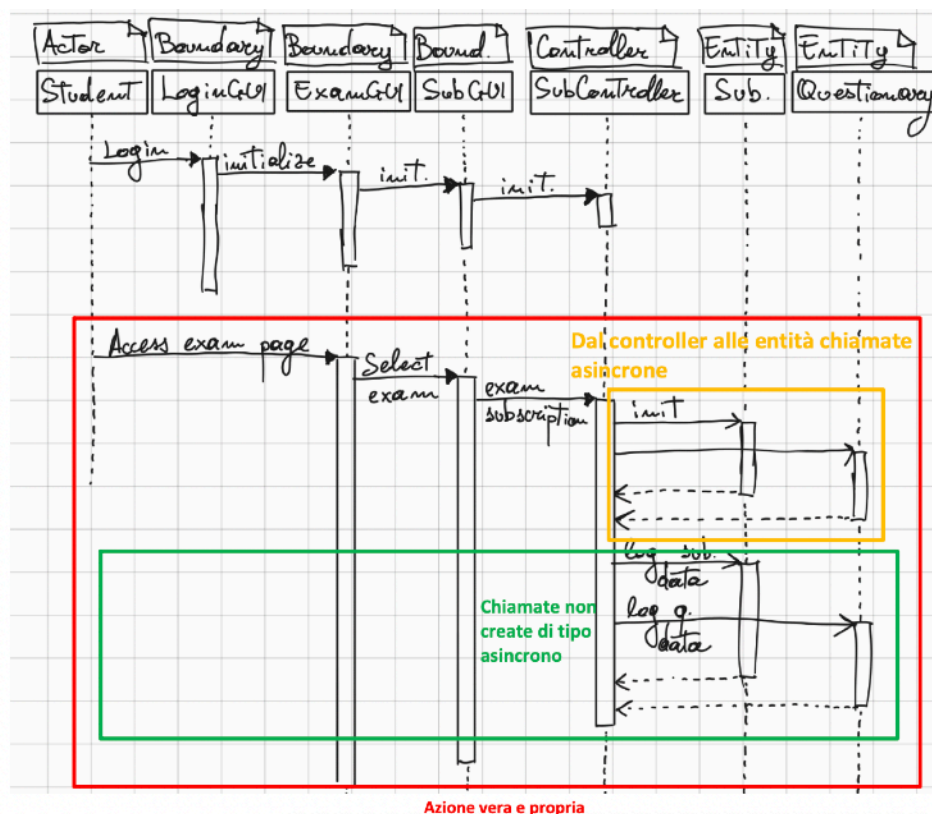
In alternativa, si può realizzare una catena di chiamate non create sincrone che parte dall'attore verso la GUI, prosegue dalla GUI al controller e dal controller alle entità tramite chiamate **create asincrono**(init).

E poi l'operazione effettiva, quindi successivamente, dal controller alle entità, vengono eseguite tutte le chiamate **non create** di tipo asincrono (log).

Di seguito entrambi gli esempi:

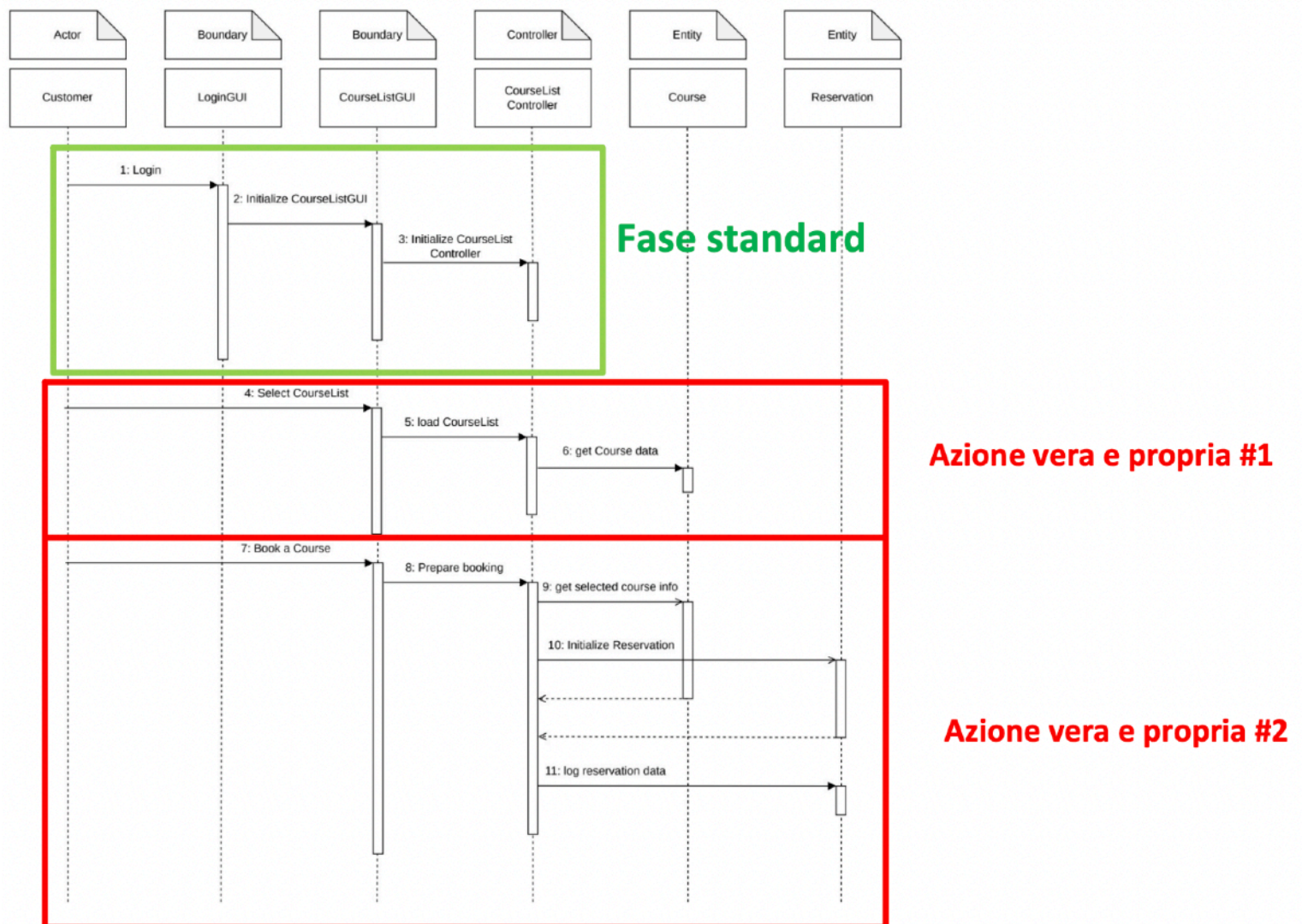


Secondo esempio



Quindi solo per la **seconda parte dell'azione vera e propria**, è importante distinguere che, in un primo momento, viene eseguita la fase di **inizializzazione(init)** tramite una serie di **chiamate sincrone o asincrone** per frecce da controller a entity.
E poi l'operazione effettiva (**nei disegni quella in verde**).

Esempio di sequence diagram completo valutato anch'esso con **30**.



Allego una prova d'esame completa relativa alla parte del Prof. D. Falessi, la quale è stata valutata con il punteggio di 30/30.

Come si può osservare, il Sequence Diagram è presente e, seguendo l'approccio di tipo algoritmico che il collega Lacroce ed io abbiamo cercato di formalizzare, l'esame è stato superato con pieno successo.

Si precisa che il presente documento non intende in alcun modo sostituirsi al materiale didattico fornito dal Professore, né tantomeno ha lo scopo di scoraggiare l'utilizzo dei materiali da lui consigliati.

L'obiettivo di questa guida è piuttosto quello di proporre un approccio di tipo più procedurale e algoritmico, in contrapposizione a quello puramente logico e creativo, che spesso genera incertezza sulla correttezza delle proprie soluzioni.

Il documento che segue è redatto in uno stile volutamente poco formale e mira a trattare direttamente i concetti fondamentali; pur non essendo da considerarsi come una fonte assoluta, le informazioni in esso contenute sono accurate, come dimostrato sia nella relazione presentata sia durante la prova d'esame, che hanno entrambe ottenuto la valutazione massima.

PROVA D'ESAME DI ISPW DEL 16 SETTEMBRE 2025

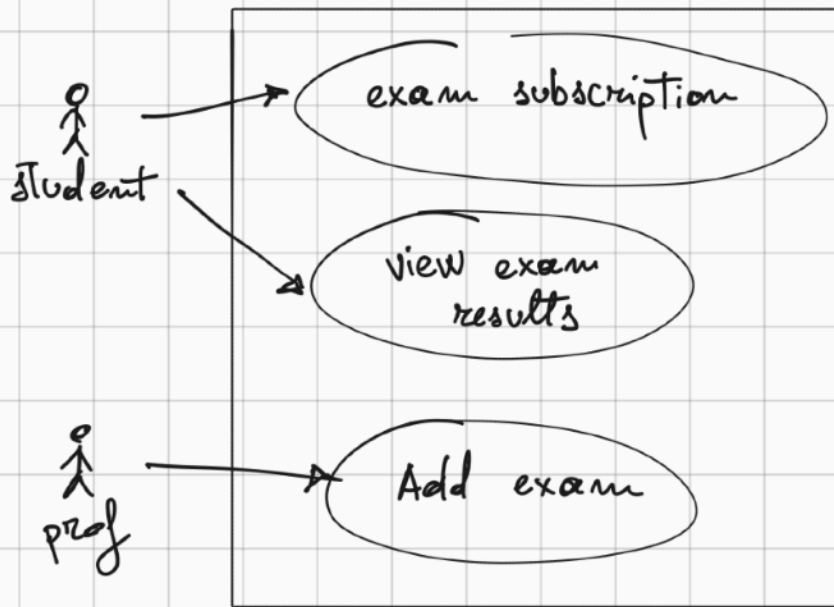
Si consideri un sistema software simile a **delphi.uniroma2.it**:



1. Si descrivano **3 requisiti funzionali**.
2. Si modelli l'**Activity Diagram** relativo ad un caso d'uso, che contenga almeno *decision, merge, fork, join* e *timer*.
3. Si modelli il **Sequence Diagram** relativo ad un caso d'uso che contenga almeno 3 *chiamate sincrone* e 3 *chiamate asincrone*.
4. Si descriva un esempio di **difetto regressivo di tipo perfettivo**, escludendo la modifica del font o simili.

Soluzione.

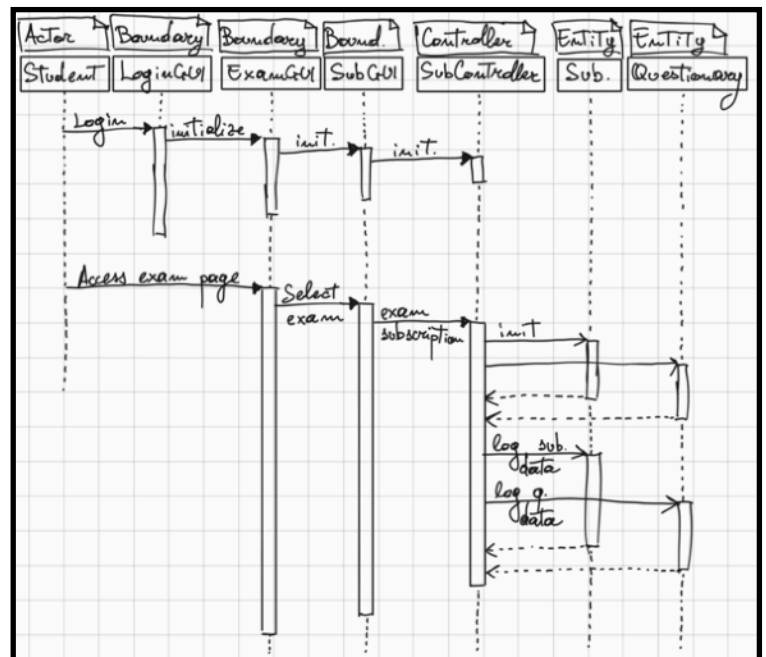
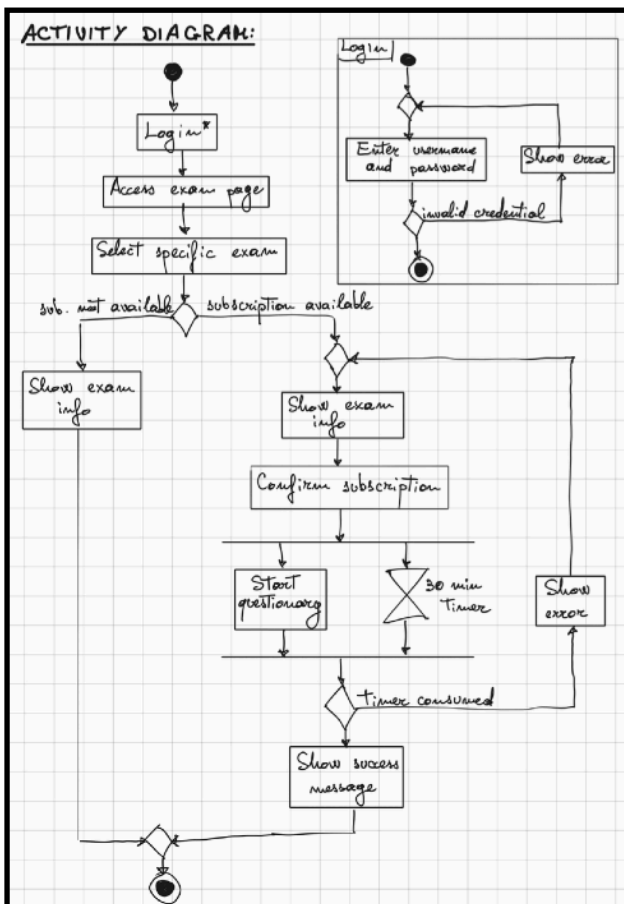
USE CASE DIAGRAM:



FUNCTIONAL REQUIREMENTS:

1. The system shall register exam subscription made by student.
2. The system shall store and show the updated list of student's exam results.
3. The system shall create new exam and shall register on the available exam list.

ACTIVITY DIAGRAM:



REGRESSIVE PERFECTIVE BUG

Un bug regressivo di tipo perfettivo può essere dovuto dalla volontà di migliorare l'esperienza d'accesso, aggiungendo l'autenticazione tramite dati biometrici (impronta digitale o riconoscimento del volto).

1. **Condizione iniziale:** Migliorare l'esperienza d'accesso.
2. **Modifiche introdotte:** Aggiunta del metodo d'accesso tramite dati biometrici (impronta digitale o riconoscimento del volto).
3. **Bug introdotto:** Alcuni dispositivi non hanno la possibilità di accedere a causa dell'assenza di dispositivi di lettura della biometria (lettore di impronta digitale o webcam per il riconoscimento del volto).
4. **Prevedibilità:** Era prevedibile, considerando che il sistema è utilizzato da dispositivi eterogenei, come dispositivi mobili datati o PC desktop senza lettore di impronta o webcam.
5. **Soluzione / Miglioramento:** Si potrebbe aggiungere un metodo di riconoscimento della mancanza dei dispositivi per la lettura dei dati biometrici ed in caso permettere l'accesso con le credenziali.

Si potrebbe altrimenti mantenere un primo accesso con le credenziali e successivamente chiedere all'utente se vuole passare al metodo d'accesso con biometria.